

Styling forms

Forms are where CSS stops being only visual polish. A form is something a user has to complete. Styling should help them understand what to fill in, which field they are on, what went wrong, and what action they can take next.

That gives this lesson a practical goal: Can the user read the form, fill it in, notice focus, spot errors, and submit it without guessing?

Work with a form that has real states

Use a support request form. It has text inputs, a select menu, a textarea, a checkbox, a submit button, a disabled button, a hint, and an error state. That mix is useful because form styling is not only about one `<input>` . Real forms have groups, labels, messages, choices, and actions.

Use this HTML:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Form styling practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Style a form people can actually use</h1>
        <p class="summary">The form should feel calm, readable, and clear in its normal,
      </section>

      <form class="support-form" action="/support" method="post">
        <div class="field">
          <label for="full-name">Full name</label>
          <input id="full-name" name="full-name" type="text" autocomplete="name" placeholder="Full name" />
        </div>

        <div class="field has-error">
          <label for="email">Email address</label>
          <input id="email" name="email" type="email" autocomplete="email" aria-invalid="true" />
          <p class="field-hint" id="email-hint">Use the email address where we can reach you</p>
          <p class="field-error" id="email-error">Enter a valid email address.</p>
        </div>

        <div class="field">
          <label for="topic">Topic</label>
          <select id="topic" name="topic">
            <option value="">Choose a topic</option>
            <option value="layout">Layout problem</option>
            <option value="forms">Form styling</option>
            <option value="accessibility">Accessibility question</option>
          </select>
        </div>
      </form>
    </main>
  </body>
</html>
```



```
<div class="field">
  <label for="message">Message</label>
  <textarea id="message" name="message" rows="5" placeholder="Tell us what happened" />
</div>

<label class="check-field">
  <input type="checkbox" name="updates" />
  <span>Send me updates about this request.</span>
</label>

<div class="form-actions">
  <button type="submit">Send request</button>
  <button class="secondary-button" type="button" disabled>Save draft</button>
</div>
</form>
</main>
</body>
</html>
```

The HTML already does important form work:

- Each main control has a `<label>` .
- `autocomplete` helps the browser suggest the right saved values for name and email fields.

- `aria-invalid` and `aria-describedby` help connect the email field to its error and hint text for assistive technology.
- The checkbox stays a real checkbox.
- The disabled button uses the real `disabled` attribute.

CSS should improve that structure, not replace it.

Give the form a readable shape

The first form problem is usually scanning. A user should be able to see where the form starts, where each field belongs, and where the action buttons are.

Add the page and form layout styles:

CSS

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f3f4f6;
}

.page {
  max-width: 720px;
  margin: 40px auto;
  padding: 0 16px;
}

.intro,
.support-form {
```

```
background-color: white;
border: 1px solid #e5e7eb;
border-radius: 8px;
}

.intro {
  margin-bottom: 24px;
  padding: 24px;
}

.support-form {
  padding: 24px;
  display: grid;
  gap: 18px;
}

.field {
  display: grid;
  gap: 6px;
}

.form-actions {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
}
```

.support-form becomes a one-column grid so the fields have consistent space between them.

.field also uses a small grid. That keeps each label, control, hint, and error message grouped together.

.form-actions uses Flexbox because the buttons are a small row of related actions. **flex-wrap: wrap** lets them move to another line if the available width gets tight.

None of this changes what the form means. It only makes the structure easier to scan.

Keep labels visible

Labels tell users what each field is for.

Placeholders can help with examples, but they are not a replacement for labels. Placeholder text disappears when the user types, and it is easy to forget what the field was asking for.

Style the labels and helper text:

CSS

```
.eyebrow {
  margin-top: 0;
  color: #2563eb;
  font-size: 14px;
  font-weight: 700;
}

h1,
p {
  margin-top: 0;
```

```
}

.summary,
.field-hint {
  color: #4b5563;
  line-height: 1.6;
}

label {
  color: #374151;
  font-weight: 700;
}

.field-hint,
.field-error {
  margin: 0;
  font-size: 14px;
}
```

font-weight: 700 makes labels easy to find while scanning.

The hint and error text use smaller type because they support the field. They should be readable, but they should not compete with the label.

 **Do not use placeholder text as the only label** - If the only label is inside the input, the user loses that label once they type. Keep a real visible label for normal form fields.

Make controls match the page

Browsers give form controls their own default styling. That is useful because the controls work immediately, but the defaults may not match your page font, spacing, or borders.

Start with the main text controls:

CSS

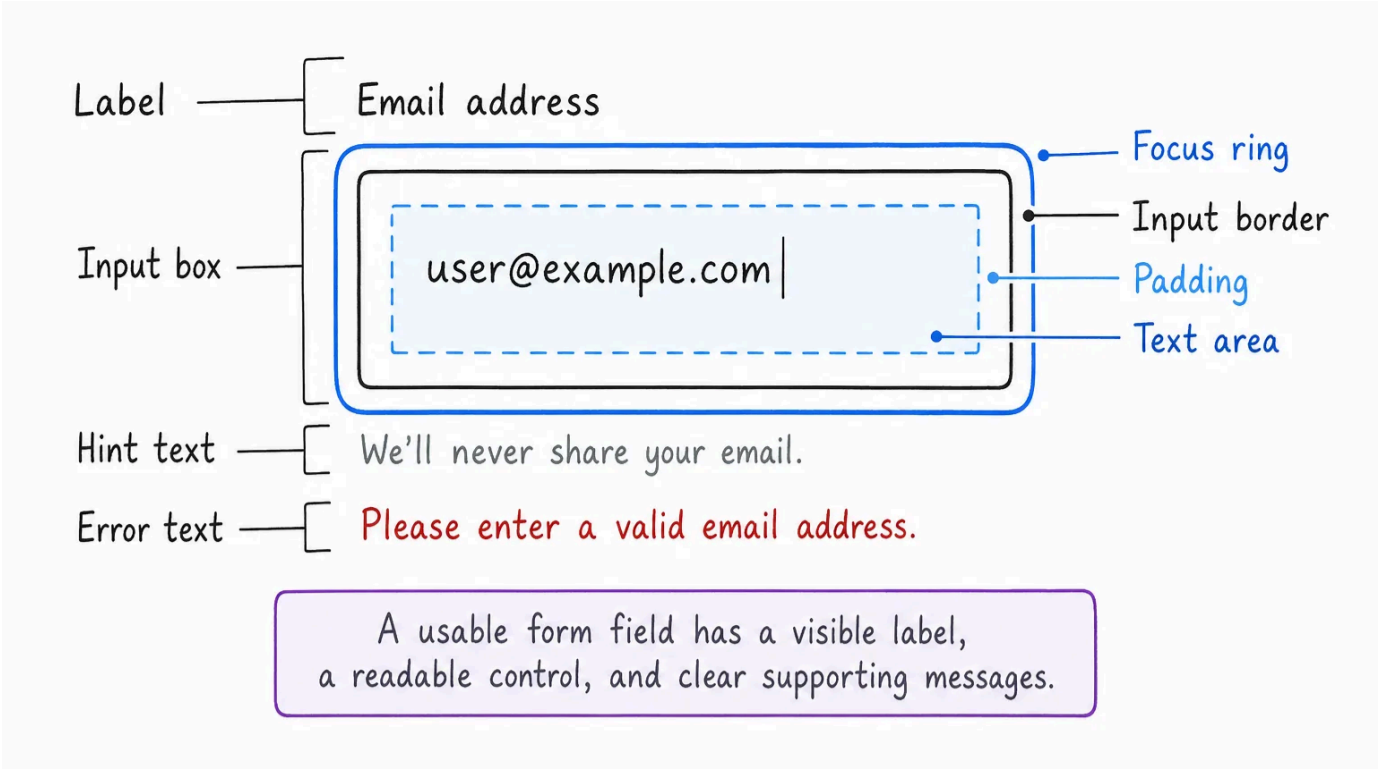
```
.field input,
.field select,
.field textarea {
  width: 100%;
  font: inherit;
  color: #111827;
  background-color: white;
  border: 1px solid #d1d5db;
  border-radius: 8px;
  padding: 10px 12px;
  line-height: 1.5;
}
```

.field input , **.field select** , and **.field textarea** target the controls that sit inside normal form fields. The checkbox is not inside **.field** , so it will not accidentally become **width: 100%** .

width: 100% makes each control fill the form column. That creates a clean edge for the user to scan.

font: inherit makes the controls use the same font as the page. Form controls often have browser-specific font defaults, so this line keeps the form visually consistent.

color controls the typed text, **background-color** controls the inside of the control, and **border** makes the editable area visible. **border-radius** softens the corners, **padding** gives text room inside the control, and **line-height** keeps typed text comfortable to read.



Let textareas grow in the right direction

A textarea is meant for longer text. It needs more room than a single-line input.

Add textarea-specific behavior:

```
CSS

.field textarea {
  min-height: 140px;
  resize: vertical;
}
```

min-height: 140px gives the message box a useful starting height.

resize: vertical lets the user make the textarea taller if they need more room. It also prevents sideways resizing, which can break the form layout.

That is a good example of form styling: you are not only changing appearance. You are preserving a useful behavior and guiding it so it fits the layout.

Keep placeholder text quiet

Placeholder text is an example, not the main instruction.

Style it so it feels secondary:

CSS

```
.field input::placeholder,
.field textarea::placeholder {
  color: #9ca3af;
}
```

::placeholder targets placeholder text inside inputs and textareas.

The color is lighter than normal text, so the user can tell it is not a real value.

Do not make placeholder text so faint that it becomes hard to read. Quiet still needs to be readable.

Make focus impossible to miss

Focus tells the user which control is active. This matters for everyone, but it is especially important for people using a keyboard. When they press Tab through the form, the focus style is their map.

Add a clear focus style:

CSS

```
.field input:focus-visible,
.field select:focus-visible,
.field textarea:focus-visible,
.check-field input:focus-visible,
button:focus-visible {
  outline: 3px solid #bfdbfe;
  outline-offset: 2px;
}

.field input:focus-visible,
.field select:focus-visible,
.field textarea:focus-visible {
  border-color: #2563eb;
}
```

:focus-visible matches when the browser decides a focus indicator should be shown, especially during keyboard navigation.

outline draws a visible ring outside the control.

outline-offset creates a little space between the control and the ring.

The second rule changes the border color only on controls that have the visible input border. That gives text fields, selects, and textareas a second focus cue without adding a meaningless border-color rule to buttons.

The checkbox and buttons are included because they can receive focus too.

⚠️ **Do not remove focus without replacing it** - Avoid `outline: none` unless you provide another clear focus style. A form without visible focus is much harder to use with a keyboard.

Show error states near the field

The email field in the HTML has `class="field has-error"` . In a real app, a validation system might add that class after checking the user's input. The class gives CSS a clear hook: style this field as a problem.

Add the error styles:

CSS

```
.field-error {
  color: #b91c1c;
  font-weight: 700;
}

.field.has-error input {
  border-color: #dc2626;
  background-color: #fef2f2;
}
```

`.field-error` makes the error text stand out.

`.field.has-error input` styles the input inside a field that has an error.

The border and background both change, so the error is not communicated by text alone and not communicated by color alone. The user sees the message and the field that needs attention.

CSS also has pseudo-classes like `:invalid` , but they can match before the user has had a fair chance to type. A class-based error state is often easier to control when you are learning or building app forms.

Style checkboxes without rebuilding them

Checkboxes and radio buttons are special controls. They have a native shape and behavior that users already understand.

For now, keep that native control and adjust the spacing and accent color:

CSS

```
.check-field {
  display: flex;
  align-items: flex-start;
  gap: 10px;
  color: #374151;
  font-weight: 500;
}

.check-field input{
  width: 18px;
```

```
height: 18px;
margin-top: 2px;
flex: none;
accent-color: #2563eb;
}
```

`.check-field` uses Flexbox because the checkbox and its text sit side by side.

`align-items: flex-start` keeps the checkbox aligned with the first line of text.

`gap: 10px` creates space between the box and the label text.

The checkbox gets a clear `width` and `height` so it is easy to click.

`flex: none` keeps the checkbox from stretching or shrinking strangely inside the row.

`accent-color` changes the native checked color without replacing the real checkbox with custom HTML.

That is a good beginner default: style the native control before trying to rebuild it.

Buttons need states too

The submit button is the main action. The disabled "Save draft" button shows that an action can exist but not be available.

Add the button styles:

CSS

```
button {
  font: inherit;
  border: 0;
  border-radius: 8px;
  padding: 10px 14px;
  font-weight: 700;
  cursor: pointer;
}

button[type="submit"] {
  color: white;
  background-color: #2563eb;
}

.secondary-button {
  color: #374151;
  background-color: #e5e7eb;
}

button:disabled {
  opacity: 0.55;
  cursor: not-allowed;
}
```


`font: inherit` keeps button text consistent with the page.

`border: 0` removes the browser's default button border because this design uses background color and shape instead.

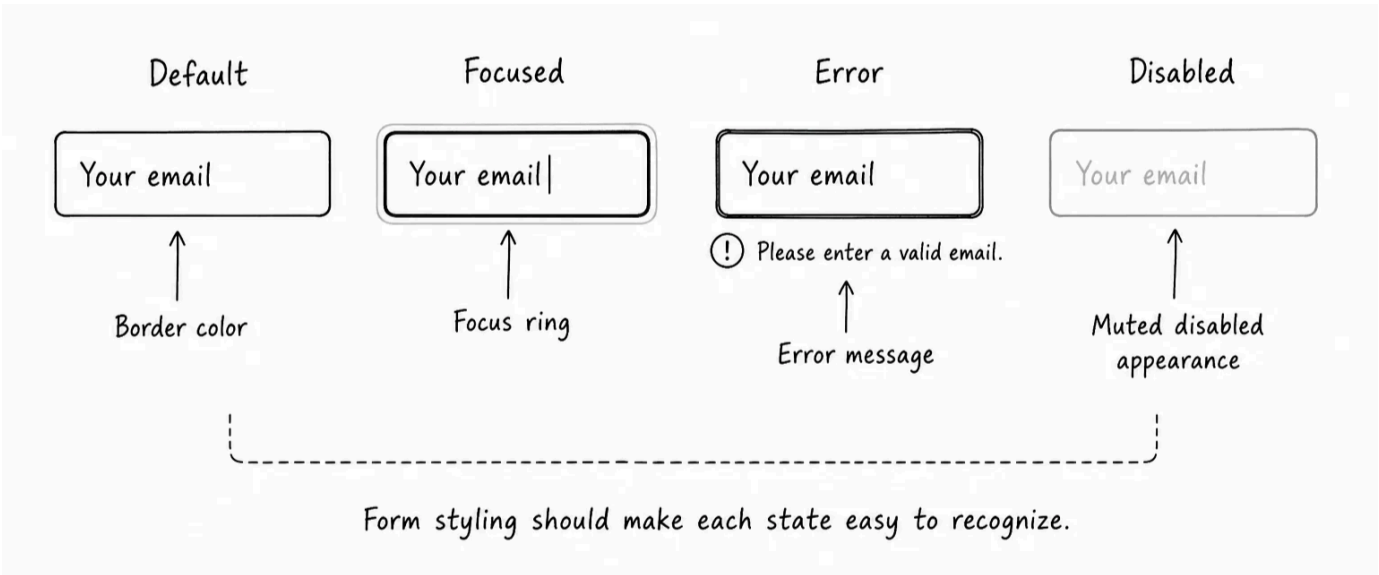
`border-radius` and `padding` make the button feel like a clickable control.

`cursor: pointer` gives mouse users a familiar cue on enabled buttons.

`button[type="submit"]` targets the main submit action.

`.secondary-button` targets the lower-priority button.

`button:disabled` styles real disabled buttons. The behavior comes from the HTML `disabled` attribute, and CSS only changes how that state looks.



Native form controls will not all look identical

Different browsers and operating systems can draw form controls slightly differently, especially select menus, date inputs, file inputs, checkboxes, and radio buttons, and that is normal.

Your first job is not to force every control to look identical everywhere. Your first job is to make the form readable, aligned, focused, and understandable while keeping the native behavior intact.

You will see CSS like `appearance: none` in custom form components. That property removes some native browser styling, but then you are responsible for rebuilding the missing visual states, so save that for later.

Inspect form states in DevTools

DevTools helps you test form CSS without guessing. Inspect an input and look at the Styles panel. Toggle padding, border, and `font: inherit` to see what each one changes.

Use the browser's force-state tools to test `:focus-visible` if your DevTools supports it. You can also press Tab through the form and watch the focus ring

move.

Inspect the email field and compare `.field.has-error input` with the normal field styles. The more specific error rule should override the normal border and background.

Inspect the disabled button and look for the `:disabled` rule. The button is disabled because of HTML; CSS only shows that state visually.

Form styling map

Here is the practical map:

Form part	CSS job
Form wrapper	Create a readable card or section around the form.
Field group	Keep label, control, hint, and error text together.
Label	Stay visible and easy to scan.
Input, select, textarea	Match the page font, spacing, border, and width.
Placeholder	Give an example without replacing the label.
Focus state	Show exactly which control is active.
Error state	Show what needs fixing near the field.
Checkbox or radio	Keep native behavior, adjust spacing and accent color.
Button	Show action priority and disabled state clearly.

The question is not "How do I make every control custom?" The better question is: "Can the user understand and complete this form?"

Try it

Use the same support form and test the states on purpose:

- Press Tab through the form. Make sure every input, select, textarea, checkbox, and button has a visible focus style.
- Remove `font: inherit` from the controls. Notice whether the form controls still match the page text.
- Remove the visible `<label>` for one field and rely only on the placeholder. Type into the field and notice why the label was important.

- Remove **has-error** from the email field. The error styling should disappear.
- Add **disabled** to the email input and inspect how the browser and your CSS treat it.
- Change **accent-color** on the checkbox and notice that the native checkbox still works.
- Increase the textarea content and resize it vertically. Check that it grows without breaking the form width.

Form styling is not about hiding the browser's controls. It is about making the controls easier to understand while preserving the behavior users depend on.

< 0 / 10 >

✔ Knew 0 Items

✧ Learnt 0 Items

⏮ Skipped 0 Items

🔄 ResetProgress

Test Your Understanding

What is the main goal of form styling?

[Click to Reveal the Answer](#)

✔ Already Know that

✧ Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →